55

BÖLÜM: KONTROL LISTELERI, YAYGIN HATALAR VE MAANG MIMAR SEVIYESI USTALIK | MODULE: CHECKLISTS, COMMON MISTAKES & MAANG ARCHITECT MASTERY

# 24 Maddelik Şartname Kontrol Listesi: Reviewed ve Approved Eşiği

The 24-Point Specification Checklist: Gatekeeper for Reviewed & Approved States

## ⚡ Akış, Varlık & Süreç Şeması

## ⚡ Process & Artifact Schematic

24 MADDELİK ŞARTNAME KONTROL LİSTESİ TABLOSU

| KATEGORİ | MADDELER |
|---|---|
| 1. Kimlik ve Yapı (Identity & Structure)<br>- Aktif fiil öbeği, kebab-case dosya adı, Tetikleyici vs Ön Koşul ayrımı | Maddeler 1 - 6<br>Sea-level hedefi, requirements.md bağı |
| 2. Ön ve Son Koşullar (Pre & Postconditions)<br>- Sistemce zorlanabilir ön koşul, asla akış içinde tekrar test edilmez | Maddeler 7 - 9<br>Başarı ve Hata minimum garantileri dolu! |
| 3. Ana Başarı Senaryosu (Main Flow)<br>- 3-9 adım, 'Top Kimde' ilk kelime, UI mekaniği (widget) temizlenmiş! | Maddeler 10 - 14<br>'Validates' kullanımı, 3 tür adımın dengesi |
| 4. Alternatif Akışlar (Alternative Flows)<br>- Her validasyona 1 akış, adıma çıpalı, kesin bitiş ('continues' / 'ends') | Maddeler 15 - 18<br>Açık uçlu akış YASAK! Alt hatalar birleştirildi |
| 5. İş Kuralları ve Referanslar<br>- BR-XXX eşleşmesi, entity_model bağı, | Maddeler 19 - 21<br>NFR ve Mockup linkleri |
| 6. Üç Okuyucu Testi (The Three-Reader Test)<br>- Zone 1 somut veriler, Kara Liste sıfır! Paydaş onaylayabilir mi? | Maddeler 22 - 24<br>Seneye stack değişse bu cümle doğru kalır mı? |

## 💡 Aptala Anlatım (ELI5 Metaforu)

## 💡 ELI5 Metaphor

### 🇹🇷 TÜRKÇE (ELI5)

Bir uzay mekiği fırlatılmadan önce astronotların ve mühendislerin elindeki kontrol listesini (checklist) düşünün: 'Oksijen vanası açık mı? Basınç tamam mı? Yakıt kapağı kilitli mi?' Tek bir madde bile işaretlenmezse roket fırlatılmaz! 24 maddelik use case kontrol listesi, kod üretim butonuna basmadan önceki o fırlatma kontrol listesidir!

### 🇬🇧 ENGLISH (ELI5)

Think of the pre-launch checklist used by NASA astronauts and flight controllers before a space shuttle liftoff: 'Oxygen valves open? Thruster pressure nominal? Fuel hatch sealed?' If a single checkbox is missed, the launch is aborted! The 24-point specification checklist is the mandatory pre-flight audit before triggering AI code generation!

## 🔍 MAANG & AIUP Seviyesi Teknik Derinlik

### 🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

24 Maddelik Şartname Kontrol Listesinin Mimarisi: Cockburn'ün klasik geçiş/kalış (pass/fail) testleri ile AI Unified Process'in katı doğrulama kurallarının birleşiminden oluşan bu liste, bir şartname 'Reviewed' ve ardından 'Approved' statüsüne geçmeden önce iki kez koşturulur. Liste 6 ana alana ayrılır: 1) Kimlik ve Yapı: İsim kullanıcının dilinde bir eylem öbeğidir, dosya adı `UC-XXX-\<kebab-name>.md` formatındadır, tek bir deniz seviyesi hedefi anlatır. 2) Ön ve Son Koşullar: Ön koşullar sistem tarafından zorlanabilir durumdadır ve akış içinde asla tekrar `if` ile test edilmez. Başarı ve hata son koşulları boşa çıkarılamaz bir sözleşmedir (no partial data). 3) Ana Akış: 3 ila 9 adımdır, her adımda topun sahibi ilk kelimedir, 'validates' kelimesi başarıyı mühürler, UI tıklama mekanikleri ayıklanmıştır. 4) Alternatif Akışlar: Her doğrulama adımının kesin bir hata akışı vardır; her akış bir adıma çıpalanır (`step 4`) ve mutlaka kesin bir dille sonlanır (`Use case continues at step N.` veya `Use case ends.`). 5) Kurallar ve Referanslar: Her BR kuralı en az bir adımdan referans alır; varlıklar `entity_model.md`'den gelir. 6) Üç Okuyucu Testi: Kara liste (JWT, SQL, bcrypt vb.) sıfır toleransla taranır; paydaş onayı ve 1 yıl sonraki teknoloji bağımsızlığı doğrulanır.

## 🔍 MAANG & AIUP Technical Deep-Dive

### 🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The 24-Point Specification Checklist Architecture: Merging Cockburn's classical pass/fail heuristics with the algorithmic verification rules of the AI Unified Process, this 24-point checklist gates specification promotion: executed once prior to 'Reviewed', and once more prior to 'Approved'. It spans six cohesive evaluation domains: 1) Identity & Structure: User-goal naming, strict kebab-case file paths matching `docs/use_cases.puml`, single-sitting sea-level scope, and requirements catalog linkages. 2) Boundary Guarantees: Preconditions must be enforceable assertions established upstream (never re-tested in-flow); failure postconditions must enforce transactional rollbacks (zero orphan state). 3) Scenario Discipline: 3-9 steps, active voice, ball-holder primacy, strict usage of 'validates', and zero UI widget micro-actions. 4) Deviation Robustness: 1-to-1 parity between validations and alternative flows, anchored triggers, and strictly banned open-ended terminations. 5) Referential Integrity: Business rules bi-directionally linked, domain nouns rooted in `docs/entity_model.md`. 6) Three-Reader Audit: Complete absence of Zone 3 blocklist terms, verifiable stakeholder outcomes, and enduring stack portability.

### ⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```
// Java 25: 24 Maddelik Şartname Denetim Motoru (Spec Linter Pipeline)
package com.maang.aiup.checklist;

import java.util.List;

public class SpecChecklistAuditor {
    public record SpecCheckResult(int passedCount, int totalCount, List<String> violations) {
        public boolean isApproved() { return violations.isEmpty() && passedCount == totalCount; }
    }

    public static class AuditorEngine {
        public SpecCheckResult auditSpec(String specContent) {
            var violations = new java.util.ArrayList<String>();

            // Kontrol 1: Kara liste kelimeleri (Zone 3)
            List<String> blocklist = List.of("JWT", "SQL", "bcrypt", "SELECT", "INSERT", "SMTP");
            for (String forbidden : blocklist) {
                if (specContent.contains(forbidden)) {
                    violations.add("Kural 22 İhlali: Yasaklı Zone 3 kelimesi tespit edildi -> " +
forbidden);
                }
            }

            // Kontrol 2: 'checks whether' kullanımı (Kural 13)
            if (specContent.contains("checks whether")) {
                violations.add("Kural 13 İhlali: 'checks whether' yerine 'validates that'
kullanılmalıdır.");
            }

            int total = 24;
            int passed = total - violations.size();
            return new SpecCheckResult(passed, total, violations);
        }
    }

    void main() {
        var engine = new AuditorEngine();
        String sampleSpec = "Step 4: System validates that user account is active (BR-001).";
        var result = engine.auditSpec(sampleSpec);

        System.out.println("✅ Şartname Denetim Skoru: " + result.passedCount() + " / " +
result.totalCount());
        System.out.println("   Onay Durumu: " + (result.isApproved() ? "APPROVED" : "REJECTED"));
    }
}
```

### ⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

#### 🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir şartnameyi 'küçük bir hata var ama kod üretirken düzeltiriz' diyerek Approved statüsüne geçirmeyin! 24 kuralın tamamı yeşil olmadan kod üretim komutunu (/implement) çalıştırmayın.

#### 🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never promote a spec to 'Approved' with known checklist violations under the assumption that AI will compensate. All 24 criteria must pass cleanly before generation.

### 🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• 24 maddelik liste şartnameyi Reviewed ve Approved aşamalarına taşıyan kapı bekçisidir.

• Liste kimlikten ön koşullara, senaryo ritminden kara liste denetimine 6 alanı kapsar.

• Tek bir ihlal bile AI'ın mimariyi yanlış üretmesine yol açabileceği için sıfır toleranslıdır.

### 🎯 Key Architectural Takeaways

• The 24-point checklist acts as the definitive gatekeeper for Reviewed and Approved states.

• It evaluates 6 domains: identity, pre/postconditions, scenario, branches, rules, and 3 readers.

• Zero tolerance: a single checklist defect invites prompt hallucination and layer misallocation.